

Problem Statement

Two players, Player 1 (Maximizer) and Player 2 (Minimizer), play a game on an $M \times N$ matrix of integers. The board contains both positive and negative scores.

Game Rules:

Player 1 starts at cell (0, 0), and Player 2 starts at (0, N-1). The starting cells are considered visited, and their values are part of the players' initial state.

Player 1 moves first. The players take turns moving their token to an unvisited adjacent cell (up, down, left, or right).

The game ends when the current player cannot make a valid move (i.e., all adjacent cells are either visited or out-of-bounds).

Player 1's score is the sum of values of all cells they have visited. Player 2 does not have their own score; their sole purpose is to make moves that minimize Player 1's final score. Player 1's goal is to maximize their final score.

Assuming both players play optimally, what is Player 1's final score?

Input Format

A 2D integer matrix (or list of lists) representing the game board.

Output Format

A single integer representing Player 1's final score, assuming optimal play from both sides.

Sample Case

Input Matrix:

```
[[1, 10],  
 [8, 2]]
```

Expected Output:

9

Explanation:

Player 1 starts at 1. Player 2 starts at 2. It's Player 1's turn.

Option A: Player 1 moves down to 8. Player 1's path is 1 -> 8, and their current score is 9. Now it's Player 2's turn. Player 2 at 2 moves to 10 to block Player 1. Now Player 1 at 8 is trapped. The game ends. Player 1's final score is 9.

Option B: Player 1 moves right to 10. Player 1's path is 1 -> 10, and their current score is 11. Now it's Player 2's turn. Player 2 at 2 moves to 8 to block Player 1. Now Player 1 at 10 is trapped. The game ends. Player 1's final score is 11.

Player 1, playing to maximize their score, anticipates both outcomes. They will choose Option B because it results in a higher score (11 vs 9).

Code

```
#include<stdio.h>
#define max 100 // Maximum grid size (100x100)

// Global variables
int m, n; // Dimensions of the board
int board[max][max]; // The board itself (values stored in each cell)
int visited[max][max]; // Keeps track of visited cells during play

// Memoization (DP) tables
int dp[max][max][max][max][2]; // Stores best score results for a given state
int calculated[max][max][max][max][2]; // Marks if a state has already been solved

// Movement directions: up, down, left, right
int dx[4] = {-1, 1, 0, 0};
int dy[4] = {0, 0, -1, 1};

// Forward declaration
int solve(int x1, int y1, int x2, int y2, int turn, int score1);

// Simple max/min helper functions
int myMax(int a, int b){
    return (a > b) ? a : b;
}
int myMin(int a, int b){
    return (a < b) ? a : b;
}

// Recursively read the matrix input and reset visited cells
void readMatrix(int i, int j){
    if(i == m) return; // End when all rows are read
    if(j == n){ // If we finish one row, move to next row
        readMatrix(i + 1, 0);
        return;
    }
    scanf("%d", &board[i][j]); // Read board value
    visited[i][j] = 0; // Mark as unvisited initially
    readMatrix(i, j + 1); // Move to next column
}

// Try all 4 possible moves for the current player
int exploreMoves(int d, int x1, int y1, int x2, int y2, int turn, int score1, int best, int *moved){
    if(d == 4){ // If all 4 directions explored, return best score
        return best;
    }

    int nx, ny; // Next move coordinates

    // Depending on whose turn it is, move player 1 or player 2
    if(turn == 0){ // Player 1's turn
        nx = x1 + dx[d];
        ny = y1 + dy[d];
    } else { // Player 2's turn
```

```

    nx = x2 + dx[d];
    ny = y2 + dy[d];
}

// Check if move is valid (inside board and not visited)
if(nx >= 0 && nx < m && ny >= 0 && ny < n && visited[nx][ny] == 0){
    *moved = 1;           // At least one move is possible
    visited[nx][ny] = 1;   // Mark new cell as visited

    int val;
    if(turn == 0){         // Player 1 tries to maximize score
        val = solve(nx, ny, x2, y2, 1, score1 + board[nx][ny]);
        best = myMax(best, val);
    }
    else{                  // Player 2 tries to minimize score
        val = solve(x1, y1, nx, ny, 0, score1);
        best = myMin(best, val);
    }

    visited[nx][ny] = 0;   // Backtrack (unmark the cell for other paths)
}

// Try the next direction
return exploreMoves(d + 1, x1, y1, x2, y2, turn, score1, best, moved);
}

// Recursive DP function to simulate the game
int solve(int x1, int y1, int x2, int y2, int turn, int score1){
    // If we already solved this state, return stored result
    if(calculated[x1][y1][x2][y2][turn] == 1){
        return dp[x1][y1][x2][y2][turn];
    }

    int best;
    int moved = 0; // Flag to check if at least one move was possible

    if(turn == 0){ // Player 1's turn (maximize score)
        best = -10000000; // Start with very low value
        best = exploreMoves(0, x1, y1, x2, y2, turn, score1, best, &moved);
        if(moved == 0){ // If no move possible → final score is current score
            best = score1;
        }
    }
    else{ // Player 2's turn (minimize score)
        best = 10000000; // Start with very high value
        best = exploreMoves(0, x1, y1, x2, y2, turn, score1, best, &moved);
        if(moved == 0){ // If no move possible → final score is current score
            best = score1;
        }
    }

    // Save result into DP and mark as calculated
    calculated[x1][y1][x2][y2][turn] = 1;
    dp[x1][y1][x2][y2][turn] = best;

    return best;
}

```

```

int main(){
    // Read board dimensions
    scanf("%d %d", &m, &n);

    // Read the board matrix values
    readMatrix(0, 0);

    // Initial positions: Player 1 starts at top-left (0,0), Player 2 at top-right (0, n-1)
    visited[0][0] = 1;    // Player 1 starting cell marked visited
    visited[0][n-1] = 1;  // Player 2 starting cell marked visited

    // Start the game simulation from initial state
    int result = solve(0, 0, m-1, n-1, 0, board[0][0]);

    // Print the final result (best score Player 1 can achieve)
    printf("%d\n", result);
    return 0;
}

```

Submission Page

Grade

Reviewed on Saturday, 6 September 2025, 12:21 AM by Automatic grade

Grade: 10.00 / 10.00

Assessment report  [-]

[+]Summary of tests

 Submitted on Saturday, 6 September 2025, 12:21 AM ( Download)

1.c

```

1  #include<stdio.h>
2  #define max 100
3
4  int m,n;
5  int board[max][max];
6  int visited[max][max];
7
8  int dp[max][max][max][max][2];
9  int calculated[max][max][max][max][2];
10
11 int dx[4] = {-1, 1, 0, 0};
12 int dy[4] = {0, 0, -1, 1};
13
14 int solve(int x1, int y1, int x2, int y2, int turn, int score1);
15
16 int myMax(int a, int b){
17     if(a > b){
18         return a;
19     }
20     else{
21         return b;
22     }
23 }
24
25 int myMin(int a, int b){
26     if(a < b){
27         return a;
28     }
29     else{
30         return b;
31     }
32 }

```